Computer Networks Lab (CNL) Experiments

Experiment Number: 01

Title: Setup a wired LAN

Problem Statement: Setup a wired LAN using Layer 2 Switch. It includes preparation of cable, testing of cable using line tester, configuration of machine using IP addresses, and testing using PING utility.

Objectives:

1. To understand the structure and working of various networks including the interconnecting devices used in them.
2. To get hands-on experience of making and testing cables.

Requirements:

- Layer-2 Switch
- 2 or more computers
- UTP (Cat5e/Cat6) cable
- RJ-45 connectors
- Crimping tool
- LAN cable tester
- Power supply

Theory:

1) What is a LAN? A Local Area Network (LAN) is a group of computers and network devices connected together, usually within the same building or campus. It allows devices to share resources like files, printers, and internet access. The most common technology used in LANs is Ethernet.

2) What is a Switch? A switch is a networking device that connects devices together on a computer network by using packet switching to receive, process, and forward data to the destination device. Unlike a hub, a Layer-2 switch forwards data only to the specific device that needs to receive it, making the network much more efficient.

3) What are the types of Cables?

- **Twisted Pair Cable:** Consists of two independently insulated wires twisted around one another. It comes in two variants: Unshielded Twisted Pair (UTP) and Shielded Twisted Pair (STP). Widely used in Ethernet networks.
- **Coaxial Cable:** Contains a central conductor, an insulating layer, a metallic shield, and an outer layer. Typically used in cable television delivery and older computer networks.
- **Fiber Optic Cable:** Uses glass or plastic threads (fibers) to transmit data as pulses of light. It offers very high bandwidth and transmits data over long distances without signal degradation.

4) Difference between IP Address and MAC Address:

Feature	**IP Address (Internet Protocol)**	**MAC Address (Media Access Control)**
Layer	Network Layer (Layer 3)	Data Link Layer (Layer 2)
Address Type	Logical Address (can change based on network)	Physical Address (hardcoded into NIC)
Length	32-bit (IPv4) or 128-bit (IPv6)	48-bit (6 bytes)
Example	`192.168.1.5`	`00:1A:2B:3C:4D:5E`
Provided by	Network Administrator / DHCP	Hardware Manufacturer

5) Procedure:

a) Preparation of Ethernet Cable:

1. Strip about 1.5 inches of the outer jacket of the UTP cable.
2. Untwist the pairs and align the wires according to the **T568B** standard (Orange-White, Orange, Green-White, Blue, Blue-White, Green, Brown-White, Brown).
3. Trim the wire ends straight across.
4. Insert the wires cleanly into the RJ-45 connector and use the crimping tool to crimp the connector tightly.

b) Testing the Cable Using Line Tester:

1. Plug both ends of the crimped cable into the LAN cable tester.
2. Turn on the tester. Ensure all 8 lights on both ends flash sequentially matching pin-to-pin, indicating successful connectivity.

c) Physical Network Setup:

1. Connect one end of the tested Ethernet cable to PC 1 and the other end to a port on the Layer-2 switch.
2. Connect PC 2 to the switch following the same process.
3. Power on the switch and the computers.

d) IP Address Configuration:

1. Go to `Control Panel > Network and Internet > Network Connections`.
2. Right-click on `Ethernet` and select `Properties`.
3. Select `Internet Protocol Version 4 (TCP/IPv4)` and assign static IP addresses (e.g., PC 1: `192.168.1.10`, PC 2: `192.168.1.11`) with Subnet Mask `255.255.255.0`.

e) Testing Using PING Utility:

1. On PC 1, open the Command Prompt (`cmd`).
2. Type `ping 192.168.1.11` and press Enter.
3. If the network is successfully set up, the terminal will display successful reply packets, indicating 0% packet loss.

Conclusion: Thus, we have successfully understood the structure and working of various networks including hands-on experience of making cables, setting them up, and verifying physical connections using diagnostic tools like PING.

Experiment Number: 02

Title: Write a Program for Error Detection and Correction for 7/8 bits ASCII codes using Hamming Code or CRC.

Objectives: To implement error detection and correction techniques.

Outcomes: Students will be able to,

1. Understand the concept of Error detection and correction techniques.
2. Implement error detection and correction using Hamming Code.
3. Implement error detection using CRC.

Theory:

1) How does Hamming Code detect and correct errors? Hamming Code is an error-correcting code used in digital transmission. It inserts redundant parity bits at positions that are powers of 2 (i.e., positions 1, 2, 4, 8...). *Example:* If sending data `1011` (4 bits), parity bits `p1, p2, p4` are added to form a 7-bit codeword. By checking the parity equations on the receiver's end, we calculate an error syndrome. If the syndrome is `000`, there's no error. If the syndrome is e.g., `110` (which is 6 in decimal), it means an error occurred at bit position 6, and the receiver flips that specific bit to correct it.

2) What is CRC? Explain with an example. Cyclic Redundancy Check (CRC) is an error-detecting code based on polynomial division. The sender appends a specific sequence of redundant bits (CRC remainder) to the end of the data so that the resulting data becomes exactly divisible by a predetermined polynomial (Generator). *Example:* Data: `100100`, Generator: `1101`. We append 3 zeros to data -> `100100000`. Dividing this by `1101` using Modulo-2 arithmetic yields a remainder (e.g., `001`). The transmitted data becomes `100100001`. The receiver divides this by `1101`. If the remainder is `000`, the data is error-free; otherwise, it is corrupted.

Code and Output:

Hamming Code (Error Detection and Correction) implementation in C++:

```
#include <iostream>
using namespace std;

int main() {
    int data[10];
    int dataatrec[10], c, c1, c2, c3, i;

    cout << "Enter 4 bits of data one by one:\n";
    cin >> data[0] >> data[1] >> data[2] >> data[4];

    // Calculate Even Parity bits
    data[6] = data[0] ^ data[2] ^ data[4];
    data[5] = data[0] ^ data[1] ^ data[4];
    data[3] = data[0] ^ data[1] ^ data[2];

    cout << "Encoded data is: ";
    for(i = 0; i < 7; i++) cout << data[i];

    // Simulating received data
    cout << "\n\nEnter received data bits one by one
    for(i = 0; i < 7; i++) cin >> dataatrec[i];

    // Error detection
    c1 = dataatrec[6] ^ dataatrec[4] ^ dataatrec[2] ^
    c2 = dataatrec[5] ^ dataatrec[4] ^ dataatrec[1] ^
    c3 = dataatrec[3] ^ dataatrec[2] ^ dataatrec[1] ^
    c = c3 * 4 + c2 * 2 + c1;

    if(c == 0) {
        cout << "\nNo error while transmission of dat
    } else {
        cout << "\nError detected at position: " << c

        cout << "\nCorrecting...\nCorrect message is:
        // Correcting the flipped bit
        dataatrec[7-c] = (dataatrec[7-c] == 0) ? 1 :

        for(i = 0; i < 7; i++) cout << dataatrec[i];
    }
    cout << "\n";
    return 0;
}
```

Output:

```
Enter 4 bits of data one by one:
1 0 1 1
Encoded data is: 1010101

Enter received data bits one by one (Introduce an error
1 0 1 0 1 1 1

Error detected at position: 2
Correcting...
Correct message is: 1010101
```

CRC Implementation in C++:

```
#include <iostream>
#include <string>
using namespace std;

string xor1(string a, string b) {
    string result = "";
    int n = b.length();
    for(int i = 1; i < n; i++) {
        result += (a[i] == b[i]) ? "0" : "1";
    }
    return result;
}

string mod2div(string divident, string divisor) {
    int pick = divisor.length();
    string tmp = divident.substr(0, pick);
    int n = divident.length();
    while (pick < n) {
        if (tmp[0] == '1') tmp = xor1(divisor, tmp) +
        else tmp = xor1(string(pick, '0'), tmp) + div
        pick += 1;
    }
    if (tmp[0] == '1') tmp = xor1(divisor, tmp);
    else tmp = xor1(string(pick, '0'), tmp);
    return tmp;
}

int main() {
    string data = "100100";
    string key = "1101";

    cout << "Original Data: " << data << "\nGenerator

    string appended_data = (data + string(key.length(
    string remainder = mod2div(appended_data, key);
    string codeword = data + remainder;

    cout << "Remainder (CRC): " << remainder << "\n";
    cout << "Encoded Transmission Data: " << codeword
    return 0;
}
```

Output:

```
Original Data: 100100
Generator Key: 1101
Remainder (CRC): 001
Encoded Transmission Data: 100100001
```

Conclusion: We have successfully implemented both Hamming Code (for error detection and correction) and the Cyclic Redundancy Check (for error detection).

Experiment Number: 03

Title: Write a Program to simulate Go Back N and Selective Repeat Modes of Sliding Window Protocol in Peer-to-Peer mode.

Objectives: To simulate Go Back N and Selective Repeat Modes of Sliding Window Protocol.

Outcomes: Students will be able to,

1. Understand the concept of Sliding Window Protocol.
2. Implement Go Back N protocol and Selective Repeat Protocol.

Theory:

1) Go Back N Protocol: In Go-Back-N (GBN), the sender can send multiple frames before receiving an acknowledgment, but is constrained to have no more than `N` unacknowledged frames in the pipeline at any given time. If the sender receives a NAK or if a timer expires for a particular frame, it must resend that frame and **all** subsequent frames in the window, discarding any newer frames received correctly by the receiver after the lost frame.

2) Selective Repeat Protocol: In Selective Repeat (SR), the sender also sends multiple frames before receiving an acknowledgment. However, if a frame is lost or damaged, the receiver buffers the subsequent frames while sending a Negative Acknowledgement (NAK) for the lost frame. The sender only retransmits the exactly lost frame, increasing efficiency over GBN, though it requires sorting and buffering logic on the receiver side.

Code and Output:

Simulation of Go-Back-N in C++:

```
#include <iostream>
#include <vector>
#include <ctime>
#include <cstdlib>

using namespace std;

void goBackN(int totalFrames, int windowSize) {
    int framesSent = 0;
    int ackReceived = 0;

    while (ackReceived < totalFrames) {
        cout << "\n--- Window Sliding ---" << endl;
        // Sending frames in window window
        for(int i = ackReceived; i < ackReceived + wi
            cout << "Sending Frame [" << i << "]" <<
        }

        // Simulate acknowledgment or failure
        int success = rand() % 3; // 1/3 chance to fa

        if (success != 0) {
            cout << "Acknowledgment received for Fram
            ackReceived++;
        } else {
            cout << "ERROR/TIMEOUT! Frame [" << ackRe
            cout << "Go-Back-N Triggered. Retransmitt
        }
    }
}

int main() {
    srand(time(0));
    cout << "Sliding Window Protocol: GO-BACK-N" << e
    goBackN(5, 3); // 5 total frames, window size 3
    return 0;
}
```

Output:

```
Sliding Window Protocol: GO-BACK-N

--- Window Sliding ---
Sending Frame [0]
Sending Frame [1]
Sending Frame [2]
Acknowledgment received for Frame [0]

--- Window Sliding ---
Sending Frame [1]
Sending Frame [2]
Sending Frame [3]
Acknowledgment received for Frame [1]

--- Window Sliding ---
Sending Frame [2]
Sending Frame [3]
Sending Frame [4]
ERROR/TIMEOUT! Frame [2] was lost.
Go-Back-N Triggered. Retransmitting entire window starti

--- Window Sliding ---
Sending Frame [2]
Sending Frame [3]
Sending Frame [4]
Acknowledgment received for Frame [2]
...
```

Simulation of Selective Repeat in C++:

```
#include <iostream>
#include <vector>
#include <ctime>
#include <cstdlib>

using namespace std;

void selectiveRepeat(int totalFrames, int windowSize)
    vector<bool> acked(totalFrames, false);
    int ackReceived = 0;

    while(ackReceived < totalFrames) {
        cout << "\n--- Transmission Round ---" << end
        int limit = min(ackReceived + windowSize, tot

        for(int i = ackReceived; i < limit; i++) {
            if(!acked[i]) {
                cout << "Sending Frame [" << i << "]"
            }
        }

        for(int i = ackReceived; i < limit; i++) {
            if(!acked[i]) {
                int success = rand() % 3; // Simulati
                if (success != 0) {
                    cout << "Acknowledgment received
                    acked[i] = true;
                } else {
                    cout << "NAK! Frame [" << i << "]
                }
            }
        }

        // Advance window
        while(ackReceived < totalFrames && acked[ackR
            ackReceived++;
        }
    }
}

int main() {
    srand(time(0));
    cout << "Sliding Window Protocol: SELECTIVE REPEA
    selectiveRepeat(4, 2);
    return 0;
}
```

Output:

```
Sliding Window Protocol: SELECTIVE REPEAT

--- Transmission Round ---
Sending Frame [0]
Sending Frame [1]
NAK! Frame [0] failed. Will selective repeat.
Acknowledgment received for Frame [1]

--- Transmission Round ---
Sending Frame [0]
Acknowledgment received for Frame [0]
```

Conclusion: We have successfully implemented and simulated the Sliding Window Protocols (Go back N protocol and Selective Repeat Protocol).

Experiment No: 04

Title: Implementation of different types of topologies and types of transmission media by using a packet tracer tool.

Objectives:

1. To learn different types of topologies and transmission media.
2. To learn the packet tracer tool.

Problem Statement: Demonstrate the different types of topologies and types of transmission media by using a packet tracer tool.

Outcomes: Students will be able to,

1. Get familiar with the packet tracer tool.
2. Implement different topologies using the packet tracer tool.
3. Understand different transmission media.

Theory:

1) Explain Cisco Packet Tracer Tool: Cisco Packet Tracer is an innovative and powerful networking simulation program that allows students to experiment with network behavior and ask "what if" questions. It provides networking simulation and visualization features that make complex networking concepts easier to understand. Students can build virtual networks using routers, switches, end devices, and cables.

2) What are the Different types of Topology? Network topology refers to the geometric arrangement of devices and cables in a local area network (LAN).

- **Bus Topology:** Every workstation is connected to a single central cable.
- **Star Topology:** All nodes are connected to a central connection point, such as a hub or a switch.
- **Ring Topology:** Devices are connected in a closed-loop circle structure.
- **Mesh Topology:** Every node is connected to every other node in the network.
- **Tree/Hierarchical Topology:** Connects multiple star topology networks using a central bus.

3) Steps to Configure and Setup a Network Topology in Cisco Packet Tracer:

Implementing Star Topology:

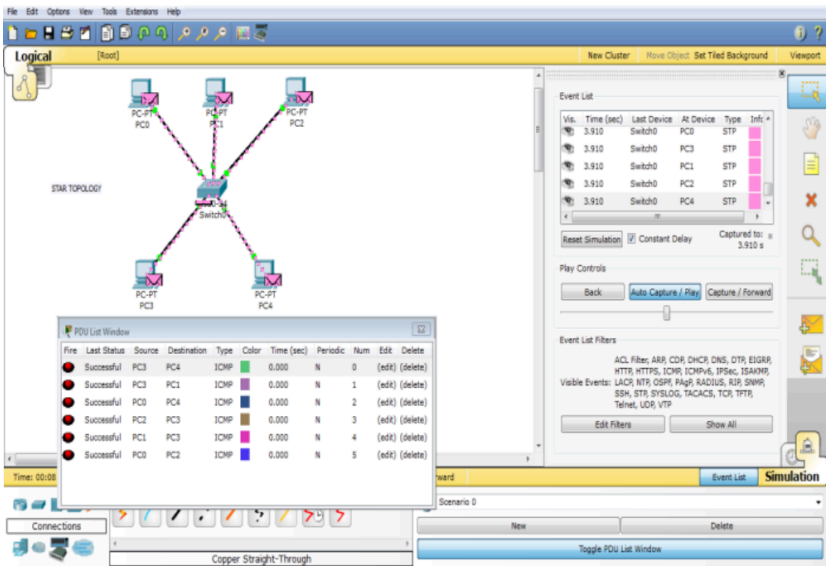
Step 1: Open the Cisco Packet Tracer desktop and select the required devices from the bottom tray.

- Take 1 `Switch (PT-Switch)`.
- Take 4 `PCs (End Devices)`.

IP Addressing Table for Configuration:

S.NO	Device	IPv4 Address	Subnet Mask
1	`pc0`	`192.168.0.1`	`255.255.255.0`
2	`pc1`	`192.168.0.2`	`255.255.255.0`
3	`pc2`	`192.168.0.3`	`255.255.255.0`
4	`pc3`	`192.168.0.4`	`255.255.255.0`

Step 2: Use the `Automatically Choose Connection Type` cable to connect every PC device directly to the central Switch, forming a Star arrangement.

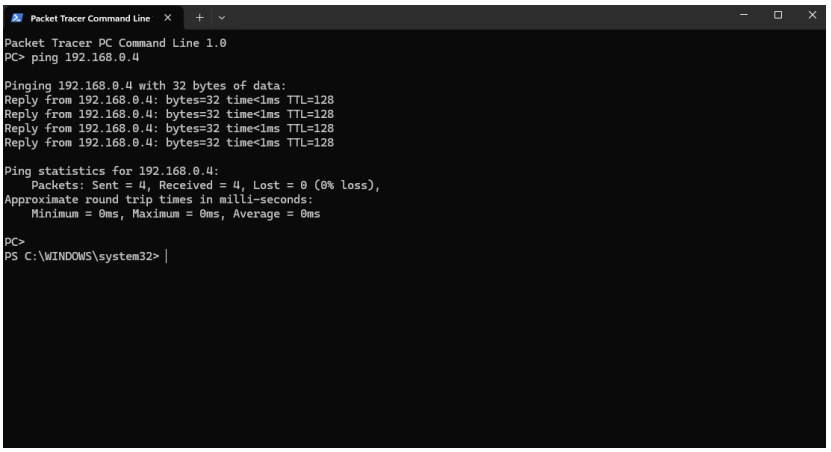


Step 3: Configure the PCs with their respective IPv4 Addresses.

- Click on `PC0` -> Go to `Desktop` tab -> Click `IP Configuration`.
- Enter IPv4 Address `192.168.0.1` and Subnet Mask `255.255.255.0`.
- Repeat this routing for `PC1`, `PC2`, and `PC3` using the respective addresses from the table above. *(Alternatively, you can open the Command Prompt terminal of the PC within Packet Tracer and type: `ipconfig 192.168.0.1 255.255.255.0`)*

Step 4: Verify the configuration and test data transmission across the network using the ping command.

- Click on `PC0` and open its `Command Prompt`.
- Execute the command: `ping 192.168.0.4` (Targeting PC3).
- If the implementation is successful, you will receive four data packet reply acknowledgments from the targeted host node without any packet loss.



Conclusion: We have successfully learned how to implement different topologies using different transmission media in the Cisco packet tracer simulation tool.